

ANÁLISE TÉCNICA: ORGANIZAÇÃO COMPUTACIONAL NO PROJETO AURORA

Introdução

O Projeto Aurora Siger é uma simulação computacional de uma missão em Marte, integrando diversos módulos de telemetria, pouso, infraestrutura e inteligência operacional. Do ponto de vista da Organização Computacional, o sistema envolve conceitos fundamentais de memória, armazenamento, fluxo de dados e barramentos, os quais são explorados nesta análise.

1. Memória

A memória primária (RAM) é utilizada pelo sistema Python para armazenar variáveis em tempo de execução, como os dicionários que representam módulos e alertas, bem como estruturas intermediárias durante o processamento. Exemplos incluem:

- O dicionário retornado por `carregar_dados_json()` que contém os módulos e alertas da colônia;
- Variáveis locais em funções como `cadastrar_modulo()` e `exibir_modulos()`;
- O cache de conteúdo fetched do módulo de IA (quando disponível).

2. Armazenamento

O armazenamento secundário é representado pelos arquivos persistentes que mantêm o estado do sistema entre execuções. O projeto utiliza duas formas de armazenamento, cada uma adequada ao tipo de dado:

- **Arquivo TXT (`registros_colonia.txt`):** utilizado para logs sequenciais de eventos, onde cada linha representa um registro com timestamp e tipo. Ideal para escrita acumulativa (modo `append`) e leitura sequencial.
- **Arquivo JSON (`dados_colonia.json`):** utilizado para dados estruturados (módulos e alertas) que precisam ser consultados, atualizados e mantidos em formato hierarchical. Permite leitura e escrita inteira do dado estruturado.

A escolha entre TXT e JSON fundamenta-se na natureza dos dados: logs são eventos livres e cronológicos, enquanto módulos e alertas possuem campos fixos que beneficiam-se da estrutura do JSON.

3. Fluxo de Dados

O fluxo de dados no sistema ocorre entre os módulos de entrada, processamento e saída. No NCAS (Fase 5), observamos o seguinte fluxo:

- **Entrada:** dados fornecidos pelo usuário via `input()` (cadastro de módulos, registros manuais, etc.) ou recebidos de outras fases (no modo pipeline integrado).
- **Processamento:** validação de dados, aplicação de regras lógicas (como `validar_alerta_critico()` e `liberar_consulta()`), e geração de prompts para IA.
- **Saída:** exibição no terminal via `print()`, gravação em arquivos TXT/JSON, e logging de eventos em `registros_colonia.txt`.

No modo integrado (`executar_fase5()`), o fluxo inclui recebimento de dados das fases 2 e 4, processamento no NCAS e envio de results para o Data Lake do `main.py`.

4. Barramentos (Buses)

Em sistemas computacionais, barramentos são vias de comunicação que transferem dados entre componentes como processador, memória e periféricos. Embora o projeto Aurora seja uma simulação de alto nível, podemos estabelecer uma analogia com os 'barramentos de software':

- O **barramento de dados** representa as chamadas de função e a passagem de parâmetros, onde dados são movidos entre módulos do código (por exemplo, de `cadastrar_modulo()` para `salvar_dados_json()`).
- O **barramento de controle** é representado pelas estruturas de controle de fluxo (if, while, try/except) que regulam quando e como as operações são realizadas.
- O **barramento de endereços** pode ser associado aos caminhos de arquivo (`ARQUIVO_REGISTROS`, `ARQUIVO_DADOS`) que indicam onde os dados devem ser lidos ou escritos.

Essa analogia ajuda a entender como a organização computacional subjacente permite a cooperação entre as partes do sistema.

Conclusão

O Projeto Aurora Siger, embora seja uma simulação, reflete princípios reais de Organização Computacional. A distinção entre memória volátil (RAM) e armazenamento persistente (disco) é clara na divisão entre variáveis em execução e arquivos TXT/JSON. O fluxo de dados segue padrões de entrada-processamento-saída, e podemos traçar paralelos entre os barramentos de hardware e os mecanismos de comunicação de software. Essa análise demonstra como conceitos de baixo nível são fundamentais mesmo em sistemas de simulação de alto nível, como aquele proposto pelo PBL da FIAP.